

LLVM Infrastructure Cheatsheet

L0 Essence: LLVM is a modern compiler infrastructure and toolchain built on a strongly-typed SSA isomorphic IR, decoupling frontend parsing, middle-end pass optimization, and backend SelectionDAG / Machine IR / register allocation.

[M1.1] Static Single Assignment (SSA) Design, Phi Nodes, and Def-Use Chains

The soul of LLVM IR is its Static Single Assignment (SSA) form: `Each Virtual Register must be assigned exactly once`. This drastically simplifies dataflow analysis and algorithms (e.g., constant folding, dead code elimination) in the middle-end. `Phi Nodes`: To represent variable choices at control flow merge points (e.g., after an `if-else` branch), LLVM introduces the `phi` instruction. It selects a value dynamically based on the predecessor basic block the execution path came from. `Def-Use & Use-Def Chains`: Every `Value` in memory tracks its users (Def-Use chain) and every `User` tracks its arguments (Use-Def chain). This enables the compiler to perform dead code detection and constant propagation in $O(1)$ time.

[M1.2] Three Isomorphic Representations of LLVM IR

LLVM guarantees that its IR preserves identical expressive power across three physical forms: `IR in Memory Class Structure`: Composed of C++ objects. The top-level container is `Module` (a compilation unit), which holds `Function` (functions), `GlobalVariable` (globals), and `BasicBlock` (basic blocks). A basic block is a container for a linear sequence of instructions. `On-Disk Bitcode (.bc)`: A compact binary encoding optimized for fast parsing and disk space efficiency. `Human-Readable Assembly Text (.ll)`: Human-readable textual representation used for debugging, optimization inspection, and verification.

[M1.3] Strongly-Typed System & Safety Verification

LLVM features a target-independent, strong type system that prevents invalid memory accesses and implicit type coercions early in the compilation flow: `Primitive Types`: Includes integers of arbitrary bit-widths (e.g., 1 for boolean, 16 for byte, 32, 164), floating-point types (`float`, `double`), and `void`. `Derived Types`: Includes arrays (`[N x Type]`), structures (`{ Type1, Type2 }`), pointers (`ptr`), and vector types (`<N x Type>`) designed for SIMD vectorization. `Safety Verification`: All type conversions must be explicit via casting instructions (e.g., `truncate`, `zero` for zero extension, `bitcast` for pointer casts); implicit truncation or coercion is strictly prohibited.

[M1.4] Basic Block CFG Constraints & Terminator Instructions

A basic block (BasicBlock) is the fundamental atomic unit of control flow in LLVM IR: `Basic Block Rules`: Satisfies the Single-Entry, Single-Exit (SESE) Control Flow Graph (CFG) constraints. No instruction other than the first can be jumped to from outside, and no instruction other than the last can branch out. `Terminator Instructions`: The final instruction in a basic block must be a terminator. It controls flow direction, such as unconditional branch `br`, conditional branch `br` with `if`, return `ret`, jump table `switch`, or exception unwind terminators like `resume/cleanupret`. `Entry Block Rule`: The first basic block of a function (Entry) must never have any CFG predecessors to ensure a deterministic entry point.

[M1.5] Stack Allocation via alloca & Memory Access Abstraction

To simplify frontend compiler construction, LLVM does not force the frontend to emit SSA form directly: `Memory Abstraction`: Frontends can allocate all local variables on the stack frame using the `alloca` instruction. `Memory Operations`: Accessing local variables is done via `load` and `store` instructions. Since stack variables reside in memory, they do not follow SSA single-assignment rules and can be written to multiple times. `Middle-End Promotion`: This creates significant memory access redundancy, which is cleaned up by the middle-end `Mem2Reg` pass, promoting memory accesses into register SSA.

[M2.1] Clang AST Traversal & IRBuilder Generation

Clang parses source code into an AST and generates IR modules using AST visitors: `Class Roles`: `Clang::CodeGen::CodeGenModule` maintains the global code generation state, `CodeGen::CodeGenFunction` manages local variable scopes, current basic blocks, and the stack frame. `IRBuilder`: Provides the `IRBuilder<>` helper. During AST traversal (e.g., visiting `Stat` and `Expr` nodes), it builds LLVM IR instructions and appends them to the current insertion point. `Constant Folding`: `IRBuilder` performs immediate constant analysis when creating instructions. If an expression can be folded (e.g., `1+2` folded to `3`), it returns the constant object directly, preventing useless instruction overhead.

[M2.2] Control Flow Branch Lowering & Basic Block Splitting

Complex control structures (e.g., `if-else`, `while/for` loops) must be lowered into flat basic blocks with jumps: `Branch Lowering`: `Split` into three new basic blocks: `then block`, `else block`, and a `merge block`. `Branch`: In the current block, compute the condition (`if`) and branch via `br` to `then`, `else`, or `merge` blocks. `Switch`: Compute the condition (`switch`) and branch via `br` to `then`, `else`, or `merge` blocks. `Short-Circuit Evaluation`: Logical operations `&&` and `||` are split into cascades of conditional jumps to achieve native short-circuiting.

[M2.3] Calling Conventions, Parameter Passing ABI, and Exception landingpad

Function calls deal with target ABI constraints and exception propagation: `ABI Lowering`: Clang translates arguments according to the target's Calling Convention (e.g., `x86_64-sysv`). Large struct parameters may be passed as pointers (byval attribute) or split into multiple integer registers. `Exception Handling`: `try-catch` blocks are lowered using specialized unwind instructions: `Calls` that might throw exceptions must be invoked using `invoke` rather than `call`. `invoke` specifies the normal return block and an unwind destination block. `The unwind block` must start with a `landingpad` instruction, declaring the types (`TypeInfo`) it can catch. Uncaught exceptions propagate via `resume`.

[M2.4] Variable Lifetimes & Dwarf Debug Metadata

Frontends retain metadata for variables and debuggers without hindering middle-end optimizations: `Lifetimes`: `Intrinsics` `lifetime.start` and `lifetime.end` mark variable scopes. Middle-end optimization passes (like `StackColoring`) reuse stack slots of non-overlapping variables, reducing stack frame sizes. `IDBInfoBuilder`: Generates metadata attached to IR nodes in Dwarf format. Metadata like `IDBInfoVariable` holds source lines and names, mapped via `llvm.dbg.declare` or `llvm.dbg.value` to track variables after SSA transformations.

[M2.5] The IR Verifier (verifyModule) for Static Correctness

Before handing IR over to middle-end passes, or immediately after a pass completes, the IR verifier must be run: `Static Verification`: `verifyModule()` performs static semantic checks across all functions and basic blocks in the Module. `Verification Criteria`: `SSA Dominance`: Any use of an SSA variable must be dominated by its Def (the execution path must pass through the Def block first). `Terminator Rule`: Every basic block must end with exactly one terminator instruction; none can appear mid-block. `Consistency`: Verifies matching types (e.g., binary `add` operands must have identical types; adding `i32` directly to `i64` is invalid).

[M3.1] NewPassManager Scheduling & Analysis/Transform Invalidation

The modern LLVM Pass Manager (NPM) maximizes pipeline efficiency: `NPM Model`: Separates passes into `AnalysisPass` (e.g., `DominatorTreeAnalysis`, queries IR without modifying it) and `TransformPass` (e.g., dead code elimination, modifies IR). `Cache`: NPM maintains an analysis cache. When a `TransformPass` runs, it returns a `PreservedAnalyses` result. `If instructions are modified` but the CFG remains unchanged, `DominatorTreeAnalysis` is preserved. `If the CFG is altered`, the cache is invalidated, and subsequent passes will trigger a lazy recalculation of the Dominator Tree.

[M3.2] The Mem2Reg Algorithm & SSA Renaming

`Mem2Reg` is the gateway pass that promotes stack-allocated `alloca` variables to registers: `Dominance Frontiers`: Analyzes variables and computes the dominance frontiers of all basic blocks containing a `store` to that variable. `Dominance frontiers` represent merge points. A `phi` node is inserted at the top of these frontier basic blocks, allocating new virtual registers. `SSA Renaming`: Traverses the Dominator Tree in a Depth-First Search (DFS), maintaining a stack of active definitions. `Replaces local load operations` with the register at the top of the definition stack; removes `store` instructions and pushes the new value onto the stack. `Cleanup`: Eliminates useless `alloca` stack memory frames, turning memory accesses into clean register dataflows.

L1 Logical Framework

`SSA Isomorphic Design & Triple Form Representation`: LLVM employs IR in Static Single Assignment form, circulating across three isomorphic states (in-memory C++ objects, on-disk bitcode, and human-readable assembly text) to ensure clean optimization decoupling. `Pass-Driven Middle-End & Alloca Promotion`: A unified `PassManager` schedules analysis and transformation passes, where the core `Mem2Reg` pass uses dominance frontiers to promote stack-based `alloca` memory variables to SSA registers. `Legalization, Selection & Machine IR Flattening`: The backend legalizes types and operations on a `SelectionDAG`, matches patterns to target physical instructions via `TableGen` rules, and flattens the graph back into linear `MachineIR`. `Register Allocation, Prolog/Epilogue & MC Emission`: Register allocators resolve infinite virtual registers onto target physical registers via interference graph coloring. PEI reconstructs physical frame layouts, and the MC layer emits standard binary object files.

L2 Data Flow Topology

`Source Code` $\rightarrow$ `Clang AST Visitor` $\rightarrow$ `llvm::IRBuilder` $\rightarrow$ `Raw LLVM IR (Alloca/Load/Store)` $\rightarrow$ `llvm::verifyModule` check $\rightarrow$ `PassManager (Mem2Reg)` $\rightarrow$ `SSA IR (Registers & Phi)` $\rightarrow$ `LICM/SimplifyCFG` passes $\rightarrow$ `Optimized IR` $\rightarrow$ `SelectionDAGISel` construction $\rightarrow$ `SelectionDAG Graph` $\rightarrow$ `Type/Operation Legalize` $\rightarrow$ `ISel Pattern Match (TableGen .td)` $\rightarrow$ `Target-Specific DAG` $\rightarrow$ `ListScheduler Scheduling` $\rightarrow$ `MachineInstr (MI)` $\rightarrow$ `Live Interval Calculation` $\rightarrow$ `RegAllocGreedy (Interference Graph Coloring)` $\rightarrow$ `Spill check? Yes` $\rightarrow$ `Insert spill load/store` $\rightarrow$ `PEI (Prologue/Epilogue)` $\rightarrow$ `MCStreamer emitter` $\rightarrow$ `MCInst Stream` $\rightarrow$ `ELF Object File / OrcJIT Execution`.

Trade-off Matrix: Optimizations & CodeGen

`Dimension: Option A` $\rightarrow$ `Option B [Trade-off / Compromise]` `Local Variable Representation: Local Stack 访问 (Alloca + Load/Store)` $\rightarrow$ `Pure SSA Register Model` [Option A simplifies frontends by avoiding variable-to-register mapping and branch phi placement, though it creates heavy load/store redundancies. Option B requires complex frontend SSA calculation, though it simplifies middle-end passes. LLVM generates `Alloca` in frontends, then immediately promotes them to SSA via `Mem2Reg` in the middle-end to combine the best of both worlds] `CodeGen Pipeline: SelectionDAG (DAG-based ISel)` $\rightarrow$ `GlobalISel (gMIR-based ISel)` [SelectionDAG uses basic-block graph structures, legalizing types and operations with high precision, but consumes heavy memory and compilation time. GlobalISel uses generic machine instructions linearly, executing global passes across basic blocks with small footprints. SelectionDAG is chosen for static release builds; GlobalISel is used for fast JIT and debug compiles] `Register Allocation: Graph Coloring (Kempe's Algorithm)` $\rightarrow$ `Greedy Allocation (RegAllocGreedy)` [Graph coloring views register mapping as an NP-complete coloring problem, providing optimal allocation for small functions but running slowly on large structures. Greedy allocation splits intervals and computes spilling cost with fast linear heuristics. LLVM defaults to Greedy allocation, optimizing compilation speed for large C++ codebases] `Cross-Module Optimization: Monolithic LTO` $\rightarrow$ `ThinLTO (Summary-based LTO)` [Monolithic LTO aggregates all bitcode into a single module, executing comprehensive inlining and dead code deletion at the expense of high link-time memory and single-threaded execution. ThinLTO runs parallel codegen by only importing relevant function IR based on summary tables. ThinLTO compromises extreme dead code deletion for multifold improvements in link time and memory consumption]

[M3.3] Loop Optimizations: LoopSimplify, LICM, and ScalarEvolution

Loops are primary optimization targets, managed by a robust suite of loop passes: `LoopSimplify`: Reconstructs loop structures to guarantee a single Preheader, a single Latch, and a dedicated Exit block, simplifying the CFG. `LICM`: Hoists instructions whose operands do not change inside the loop body (and have no side effects) are guaranteed to execute on loop exit into the Preheader to prevent redundant execution. `ScalarEvolution`: Formulates closed-form expressions for induction variables (e.g., loop index i). SCEV represents variables as polynomials like $\{Start, +, Step\}_L$ to compute loop trip counts and align vectorization.

[M3.4] Redundancy Elimination: EarlyCSE, ADCE, and GVN

Eliminating redundant calculations is the most direct way to improve execution speed: `EarlyCSE` (Common Subexpression Elimination): Performs a pre-order traversal of the Dominator Tree, maintaining a hash table of previously computed expressions (e.g., $X_3 = add\ i32\ Y_1, Z_2$). Sub-nodes with identical operands are replaced with Z_3 , avoiding recomputation. `ADCE` (Aggressive Dead Code Elimination): Assumes all instructions are dead unless they have side effects (e.g., writing to memory, returns, syscalls). Traces backward along Use-Def chains from side-effect roots, preserving only active dependency chains and erasing the rest. `GVN` (Global Value Numbering): Assigns a unique global value number to all expressions. Unlike EarlyCSE, it can determine equivalence across control flow branches by analyzing value graphs.

[M3.5] SimplifyCFG & The Inliner

Simplifying control flow graphs and reducing calling overheads occur frequently in the middle-end: `SimplifyCFG`: Merges basic blocks that have a single predecessor/successor. `Deletes` unreachable basic blocks. `Promotes` branches with a single phi (e.g., if-then-else returning constants) to a branchless select instruction, leveraging hardware conditional moves. `Inliner`: Calculates an Inline Cost based on the size of the target function, call frequency (e.g., loop nesting), and branch hotness. `Inline` If the cost falls below a threshold, the callee's IR is copied directly into the caller, eliminating stack parameter pushing, calls, and return overheads while enabling cross-function optimizations.

[M4.1] CodeGen Pipeline Stages

The LLVM backend (CodeGen) acts as a multi-stage factory transforming target-independent IR to target-specific assembly: `SelectionDAGISel`: Builds a target-independent SelectionDAG representation from LLVM IR. `ISel`: Maps DAG nodes to target physical instructions via pattern matching. `Instruction Selection (ISel)`: Maps DAG nodes to target physical instructions via pattern matching. `Instruction Scheduling`: Orders DAG nodes into linear MachineIR sequences. `Register Allocation (RegAlloc)`: Maps infinite virtual registers to physical registers. `PEI & MC Emission`: Constructs stack frame prologues/epilogues and writes output assembly or binary objects.

[M4.2] SelectionDAG Construction & Legalization

SelectionDAG represents data and control dependencies within a basic block as a Directed Acyclic Graph: `SDNode` represent operations, edges (`SDValue`) represent data flow. A special "Chain Edge" enforces ordering on side-effect instructions (like Load/Store) to prevent invalid reordering. `Type Legalization`: Zero-extends unsupported narrow types (e.g., 18 to 32-bit if the hardware only supports 32-bit registers). `Split`: Divides wide types into supported widths (e.g., splitting 164 into two 32-bit targets). `Legalization`: Replaces unsupported operations with runtime library calls (e.g., replacing `sddiv` with a call to `_divdi3` if the target CPU lacks hardware division).

[M4.3] TableGen-Driven Instruction Selection (ISel)

Instruction Selection maps target-independent SelectionDAG nodes to actual machine instructions: `TableGen`: Developers define declarative pattern matches in target description (.td) files (e.g., defining an `x86 ADD32rrz` matching DAG (`add i32: $src1, i32: $src2`)). `BuildTime Compilation`: The TableGen compiler (`llvm-tblgen`) parses .td files into a compact byte-mapped pattern matcher table. `Runtime Matching`: The ISel engine runs a state machine matching DAG nodes against the table, replacing generic `SDNode` instances with physical instruction nodes.

[M4.4] Instruction Scheduling & MachineInstr Flattening

SelectionDAGs must be linearized before register allocation and emission: `Constrain`: SelectionDAGs are graph structures, whereas physical execution pipelines are linear. `Scheduling`: Schedulers (e.g., `ScheduleMILive`) use heuristics (considering pipeline latencies, register pressure, etc.) to traverse the DAG, producing an ordered sequence that minimizes CPU stalls and limits register lifetimes. `MachineInstr (MI)`: The scheduler deconstructs the DAG back into a linear doubly-linked list of `MachineInstr` objects, reintroducing virtual registers.

[M4.5] GlobalISel: Fast Pipeline via Generic Machine IR (gMIR)

To resolve the high memory overhead and execution speed bottlenecks of SelectionDAG's multiple legalization phases, LLVM introduced GlobalISel: `GenericMIR`: Bypasses SelectionDAG entirely, translating IR directly to generic machine instructions (e.g., `%vreg0 = G_ADD i32 %vreg1, %vreg2`), preserving a linear MIR representation throughout. `GlobalPasses`: Legality, RegBankSelect, and InstructionSelect are decoupled into standard `MachineFunctionPass` classes. They analyze code across basic blocks, offering superior global optimization opportunities. `Efficiency`: Minimizes memory allocation overhead, yielding compilation throughput increases of several fold, ideal for JIT engines and debug builds.

[M5.1] MachineIR (MIR) Structure, Virtual/Physical Registers

MachineIR is the final representation container, matching physical assembly syntax while maintaining virtual registers before register allocation: `MachineHierarchy`: Outer wrapper is `MachineFunction`, which contains `MachineBasicBlock (MBB)` and `MachineInstr (MI)`. `MachineOperand`: Operands inside instructions can be virtual registers (`%stack_val1`), physical registers (`$rax`), immediates, or memory symbols. `RegisterClass`: Virtual registers must bind to a register class (e.g., `x86 GR32` for 32-bit general-purpose registers), which defines the pool of physical registers available for allocation.

[M5.2] The RegAllocGreedy Coloring & Spilling Algorithm

LLVM employs a greedy register allocator (RegAllocGreedy) to map virtual registers to physical registers: `Live Intervals`: Computes the instruction range (definition to last use) for which each virtual register remains active. `Interference Graph`: Builds an interference graph where nodes are intervals and edges connect overlapping intervals (meaning they cannot share a physical register). `Coloring & Allocation`: Sorts virtual registers by priority (shorter intervals and loop-nest variables have higher priority) and assigns free physical registers. `Spilling`: If physical registers are exhausted, a "Spill" is triggered. `Spill Insertion`: Chooses the lowest-cost spill candidate, allocates a stack slot, and inserts a store after its definition and a load before its uses, splitting the live interval for reallocation.

[M5.3] PEI Frame Layout Reconstruction

After register allocation, all virtual registers are replaced by physical ones, but the stack frame layout is not yet complete: `PrologueEpilogueInserter (PEI)`: Computes the final size of the stack frame (aggregating spill slots and local alloca arrays). `Determines` stack alignment (e.g., `x86_64` mandates 16-byte aligned stack pointers). `Physical Insertion`: Inserts Prologue assembly (e.g., adjusting SP subq \$32, %rsp and saving FP) at the function entry basic block. `Inserts` Epilogue assembly (restoring SP and callee-saved registers) before all function return ret blocks. `Callee-Saved Registers`: The ABI specifies registers that must be preserved (e.g., `%rbx, %r12`). If modified, PEI pushes them in the Prologue and pops them in the Epilogue.

[M5.4] TableGen .td Target Descriptions

TableGen is the core mechanism enabling LLVM to easily extend to new hardware architectures: `TableGen Language`: A strongly-typed, record-oriented description language (.td). `Description Focus`: Declares physical registers, names, aliases, and overlapping sub-registers (e.g., `EAX` as the low 32 bits of `RAX`). `Instruction Sets`: Declares instruction mnemonics (add), binary instruction encoding formats, operand input/output types, latencies, and implicit register side effects. `Calling Conventions`: Defines register priority for argument passing (e.g., `%rdi, %rsi, %rdx` on `x86_64`) and stack offset rules. `CodeGen Auto-generation`: During build time, `llvm-tblgen` compiles .td files into C++ pattern matching tables and type-helper code loaded into CodeGen.

[M6.1] MC Layer: MCStreamer & MCInst Instructions

The MC (Machine Code) layer acts as the final assembler/disassembler engine: `MCInst`: Unlike `MachineInstr`, MCInst is stripped of compiler-specific control flow and virtual registers, holding only physical register IDs, immediates, and opcode bytes. `MCStreamer`: Provides a uniform streaming interface. It selects `MCStreamer` to write text assembly, or `MCObjectStreamer` to forward the stream to the binary encoder. `Assembly & Disassembly`: The encoder serializes instructions to machine code; the disassembler reads binary arrays, builds an `MCInst` tree via table-mapped decoders, and outputs text.

[M6.2] Object File Emission: ELF/Mach-O Sections and Relocation Tables

The MC ObjectWriter packages instruction streams into system-executable object files: `Section Generation`: Converts code into the .text binary section, read-only constants into .rodata, and global variables into .data. `Symbol Tables`: Extracts function and global variable symbols into a .symtab section, recording byte offsets within their parent sections. `Relocations`: Calls to external functions (e.g., printf) or accesses to global variables have unknown absolute addresses during single-module compilation. The compiler fills these sites with placeholders and writes relocation records into .rela.text instructing the linker how to patch the addresses (e.g., via PC-relative `R_X86_64_PC32` relocations).

[M6.3] OrcJIT Engine & JITLink Linker

LLVM's JIT (Just-In-Time) engine compiles IR dynamically into memory for immediate execution: `OrcJIT Architecture`: Employs a layered architecture. The high-level `LLJIT` interface sits atop an `IRCompilerLayer` (compiles IR to objects) and an `ObjectLinkingLayer` (manages memory allocation). `On-Demand Compilation`: OrcJIT registers modules lazily. Actual compilation and machine code generation are deferred to background worker threads until a function's symbol is first resolved. `JITLink`: A modern linker replacing the legacy `RuntimeDyld`. It parses object files in memory and replicates operating system linking: allocating `r-x` memory for code and `r--` for data, executing relocations directly into active memory buffers, and transferring control.

[M6.4] Link-Time Optimization (LTO) & ThinLTO

LLVM pioneered Link-Time Optimization (LTO) at scale: `Limitation`: Traditional compilers optimize single modules, leaving linkers to copy symbols without cross-module inlining. `Monolithic LTO`: Compiles source files to LLVM IR bitcode instead of assembly. During linking, the LTO driver reads all bitcode into a single massive memory module, executing global optimizations, though at high memory and time costs. `ThinLTO`: Emits thin bitcode containing local IR alongside a lightweight summary table (storing function size, import/export relations). `ThinLTO Phase 2`: During linking, the ThinLTO driver loads summaries globally. If module A needs to inline a function in B, it imports only that function's IR, allowing optimization passes to run fully in parallel across cores with low memory overhead.

Zone T1: LLVM CLI Commands & Debug Flags

`clang++ -O3 -mllvm -print-after-all main.cpp` : Prints the complete IR text

after every single middle-end pass, allowing developers to isolate which pass introduced an optimization bug. `clang++ -Xclang -ast-dump -fsyntax-only main.cpp` : Outputs Clang's Abstract Syntax Tree (AST) to standard output, verifying parser correctness before lowering to LLVM IR. `opt -passes='mem2reg,simplifycfg' -S input.ll -o output.ll` : Runs the new Pass Manager (NPM) stand-alone, applying the specified pass pipeline (promoting alloca to SSA and simplifying control flow) to output textual .ll IR. `llvm-cpp2x86-64 -scpu=skylake -print-after-isel input.ll` : Compiles LLVM IR to x86 assembly, printing out target machine instructions immediately after Instruction Selection (ISel) is resolved.

Zone T2: LLVM IR Assembly Patterns & SSA Features

`Stack Allocation` vs. `SSA Form Promotion`: `Unoptimized IR` stores local variables (e.g., `int a = 5;`) on the stack: `%a = alloca i32, align 4` ; Allocates 4 bytes on stack frame `%store i32 5, ptr %a, align 4` ; Stores 5 into local stack memory `%i = load i32, ptr %a, align 4` ; Loads stack memory back to virtual register `%i` `%add = add nsw i32 %i, 10` ; Performs addition `After running Mem2Reg optimization`, stack accesses are removed: `%i`; `alloca`, `store`, `load` instructions are completely erased `%add = add nsw i32 5, 10` ; Constant propagated directly to SSA instruction `Phi Node CFG Convergence`: `When code paths merge`, the phi instruction resolves multiple definition inputs: `when: %val_then = add i32 %x, 1` `label %merge` `%val_else = sub i32 %x, 1` `label %merge` `phi syntax`: selects register based on the actual predecessor block executed `%val = phi i32 [ %val_then, %then ], [ %val_else, %else ]` `ret i32 %val`